

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

Question: Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Program

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(

    p_fromAccount NUMBER,

    p_toAccount NUMBER,

    p_amount NUMBER

)

IS

    v_balance NUMBER;

BEGIN

    SELECT Balance

    INTO v_balance

    FROM Accounts

    WHERE AccountID = p_fromAccount;

    IF v_balance < p_amount THEN

        RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance!');

    END IF;

    UPDATE Accounts

    SET Balance = Balance - p_amount

    WHERE AccountID = p_fromAccount;
```

```
UPDATE Accounts
```

```
SET Balance = Balance + p_amount
```

```
WHERE AccountID = p_toAccount;
```

```
COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('Fund transfer completed successfully.');
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
ROLLBACK;
```

```
DBMS_OUTPUT.PUT_LINE('Transfer failed: ' || SQLERRM);
```

```
END;
```

Scenario 2: Manage errors when updating employee salaries.

Question: Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Program

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
```

```
    p_employeeID NUMBER,
```

```
    p_percentage NUMBER
```

```
)
```

```
IS
```

```
BEGIN
```

```
UPDATE Employees

SET Salary = Salary + (Salary * p_percentage / 100)

WHERE EmployeeID = p_employeeID;

IF SQL%ROWCOUNT = 0 THEN

    RAISE NO_DATA_FOUND;

END IF;

COMMIT;


DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
```

EXCEPTION

```
    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('Employee ID not found.');
```

WHEN OTHERS THEN

```
        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
```

Scenario 3: Ensure data integrity when adding a new customer.

Question: Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

Program

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(

    p_customerID NUMBER,
```

```

    p_name VARCHAR2,

    p_dob DATE,

    p_balance NUMBER

)

IS

BEGIN

    INSERT INTO Customers

VALUES(

    p_customerID,

    p_name,

    p_dob,

    p_balance,

    SYSDATE

);

COMMIT;

DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
```



```

EXCEPTION

    WHEN DUP_VAL_ON_INDEX THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Customer ID already exists.');
```



```

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

END;